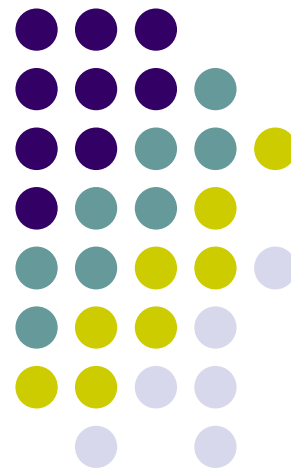


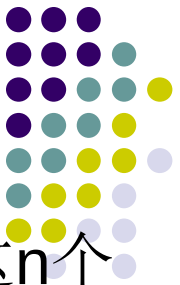
第2章 递归与分治策略 (二)

曾华琳





线性时间选择



线性时间选择

给定线性序集中 n 个元素和一个整数 k , $1 \leq k \leq n$, 要求找出这 n 个元素中第 k 小的元素

```
template<class Type>
Type RandomizedSelect(Type a[],int p,int r,int k)
{
    if (p==r) return a[p];
    int i=RandomizedPartition(a,p,r),
        j=i-p+1;
    if (k<=j) return RandomizedSelect(a,p,i,k);
    else return RandomizedSelect(a,i+1,r,k-j);
}
```

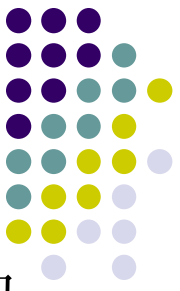
在最坏情况下, 算法**randomizedSelect**需要 $O(n^2)$ 计算时间
但可以证明, 算法**randomizedSelect**可以在 $O(n)$ 平均时间内
找出 n 个输入元素中的第 k 小元素。



线性时间选择

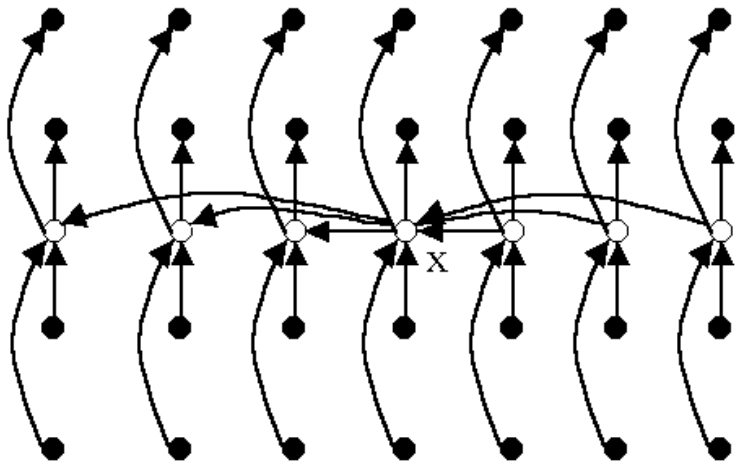
如果能在线性时间内找到一个划分基准，使得按这个基准所划分出的2个子数组的长度都至少为原数组长度的 ε 倍($0 < \varepsilon < 1$ 是某个正常数)，那么就可以**在最坏情况下**用 $O(n)$ 时间完成选择任务。

例如，若 $\varepsilon=9/10$ ，算法递归调用所产生的子数组的长度至少缩短 $1/10$ 。所以，在最坏情况下，算法所需的计算时间 $T(n)$ 满足递归式 $T(n) \leq T(9n/10) + O(n)$ 。由此可得 $T(n) = O(n)$ 。

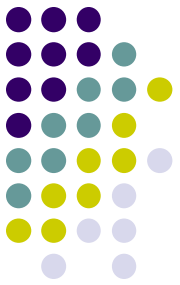


线性时间选择

- 将 n 个输入元素划分成 $\lceil n/5 \rceil$ 个组，每组5个元素，只可能有一个组不是5个元素。用任意一种排序算法，将每组中的元素排好序，并取出每组的中位数，共 $\lceil n/5 \rceil$ 个。
- 递归调用**select**来找出这 $\lceil n/5 \rceil$ 个元素的中位数。如果 $\lceil n/5 \rceil$ 是偶数，就找它的2个中位数中较大的一个。以这个元素作为划分基准。



设所有元素互不相同。在这种情况下，找出的基准 x 至少比 $3(n-5)/10$ 个元素大，因为在每一组中有2个元素小于本组的中位数，而 $n/5$ 个中位数中又有 $(n-5)/10$ 个小于基准 x 。同理，基准 x 也至少比 $3(n-5)/10$ 个元素小。而当 $n \geq 75$ 时， $3(n-5)/10 \geq n/4$ 所以按此基准划分所得的2个子数组的长度都至少缩短 $1/4$ 。



```
Type Select(Type a[], int p, int r, int k)
```

```
{
```

```
    if (r-p<5) {
```

```
        用某个简单排序算法对数组 a[p..r] 排序
```

```
        ret
```

```
    };
```

```
    for (
```

```
        将
```

```
        与
```

复杂度分析

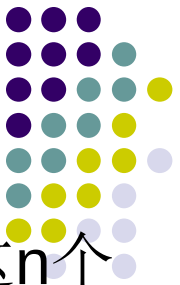
$$T(n) \leq \begin{cases} C_1 & n < 75 \\ C_2 n + T(n/5) + T(3n/4) & n \geq 75 \end{cases}$$

$$T(n) = O(n)$$

```
//找中位数的中位数， r-p-4即上面所说的n-5
```

上述算法将每一组的大小定为5，并选取75作为是否作递归调用的分界点。这2点保证了 $T(n)$ 的递归式中2个自变量之和 $n/5 + 3n/4 = 19n/20 = \epsilon n$ ， $0 < \epsilon < 1$ 。这是使 $T(n) = O(n)$ 的关键之处。当然，除了5和75之外，还有其他选择。

```
}
```



线性时间选择

给定线性序集中 n 个元素和一个整数 k , $1 \leq k \leq n$, 要求找出这 n 个元素中第 k 小的元素

```
template<class Type>
Type RandomizedSelect(Type a[],int p,int r,int k)
{
    if (p==r) return a[p];
    int i=RandomizedPartition(a,p,r),
        j=i-p+1;
    if (k<=j) return RandomizedSelect(a,p,i,k);
    else return RandomizedSelect(a,i+1,r,k-j);
}
```

在最坏情况下, 算法**randomizedSelect**需要 $O(n^2)$ 计算时间
但可以证明, 算法**randomizedSelect**可以在 $O(n)$ 平均时间内
找出 n 个输入元素中的第 k 小元素。



线性时间选择

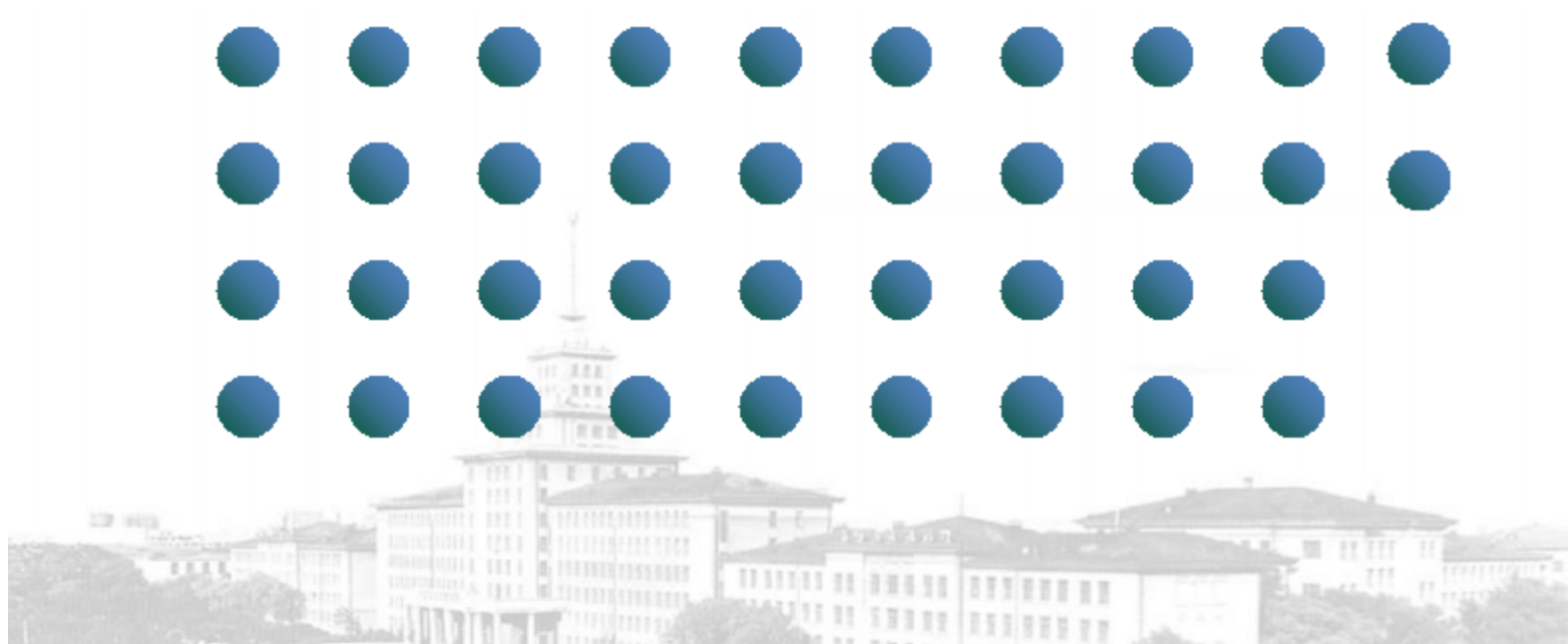
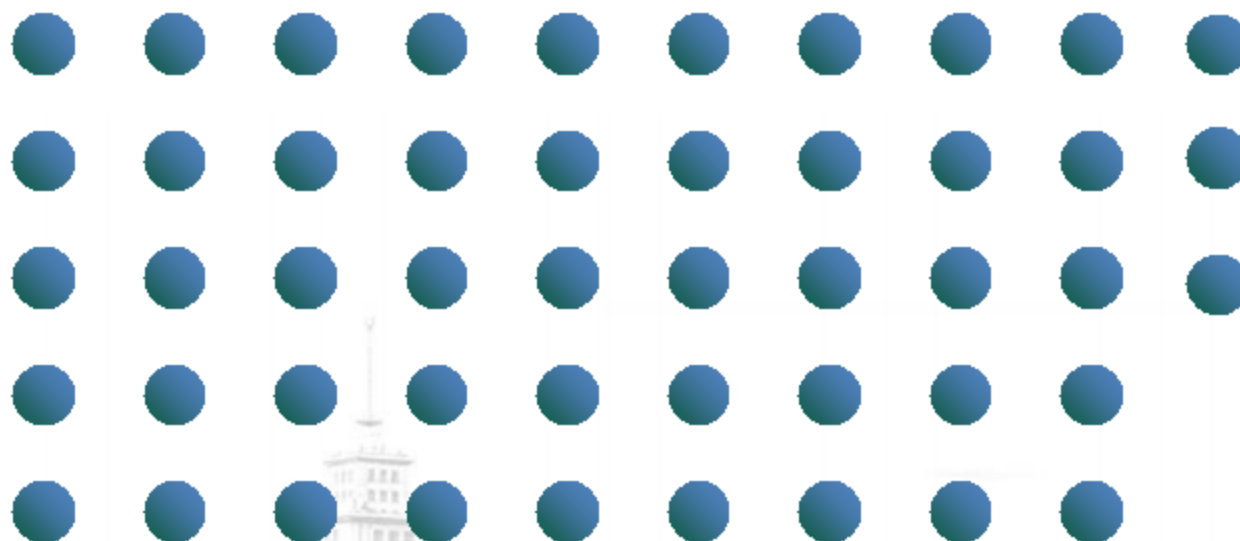
如果能在线性时间内找到一个划分基准，使得按这个基准所划分出的2个子数组的长度都至少为原数组长度的 ϵ 倍($0 < \epsilon < 1$ 是某个正常数)，那么就可以在**最坏情况下**用 $O(n)$ 时间完成选择任务。

例如，若 $\epsilon=9/10$ ，算法递归调用所产生的子数组的长度至少缩短 $1/10$ 。所以，在最坏情况下，算法所需的计算时间 $T(n)$ 满足递归式 $T(n) \leq T(9n/10) + O(n)$ 。由此可得 $T(n) = O(n)$ 。



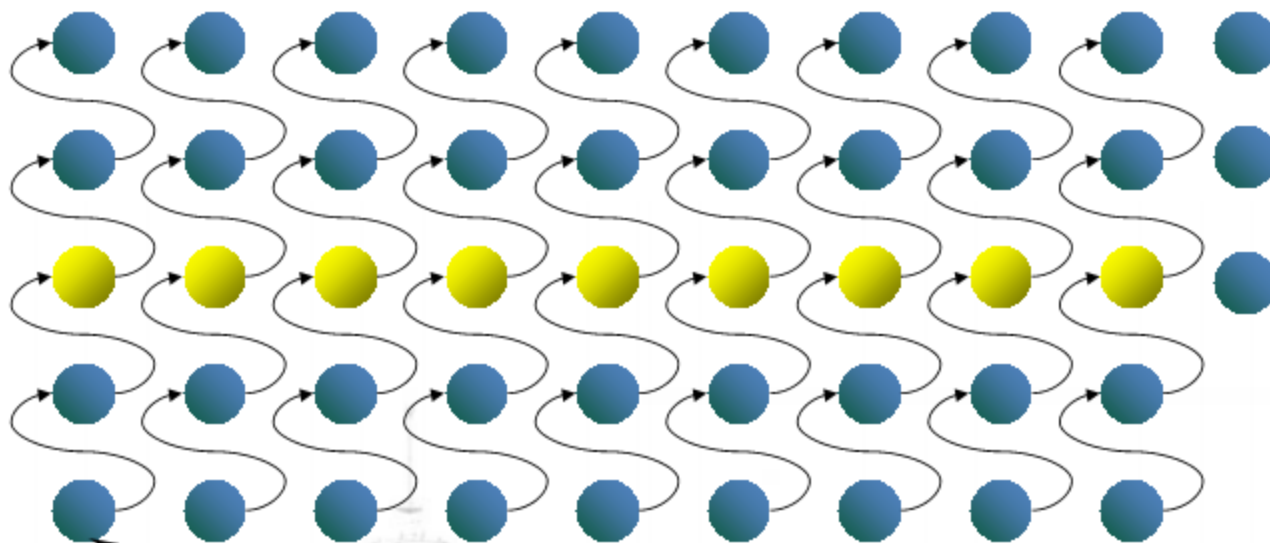
求解步骤

第一步： 分组，每组5个数
最后一组可能少于5个数





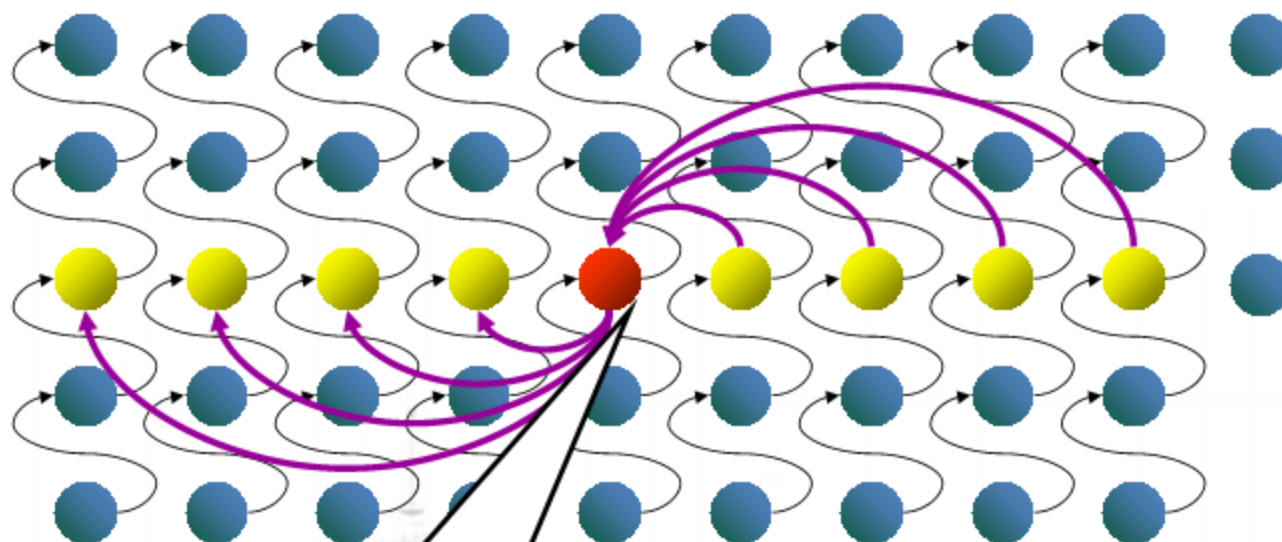
第二步：将每组数分别用InsertionSort排序 选出每组元素的中位数



排序每组数时，比较操作的次数为 $5(5-1)/2=10$ 次
总共需要 $10*\lceil n/5 \rceil$ 次比较操作



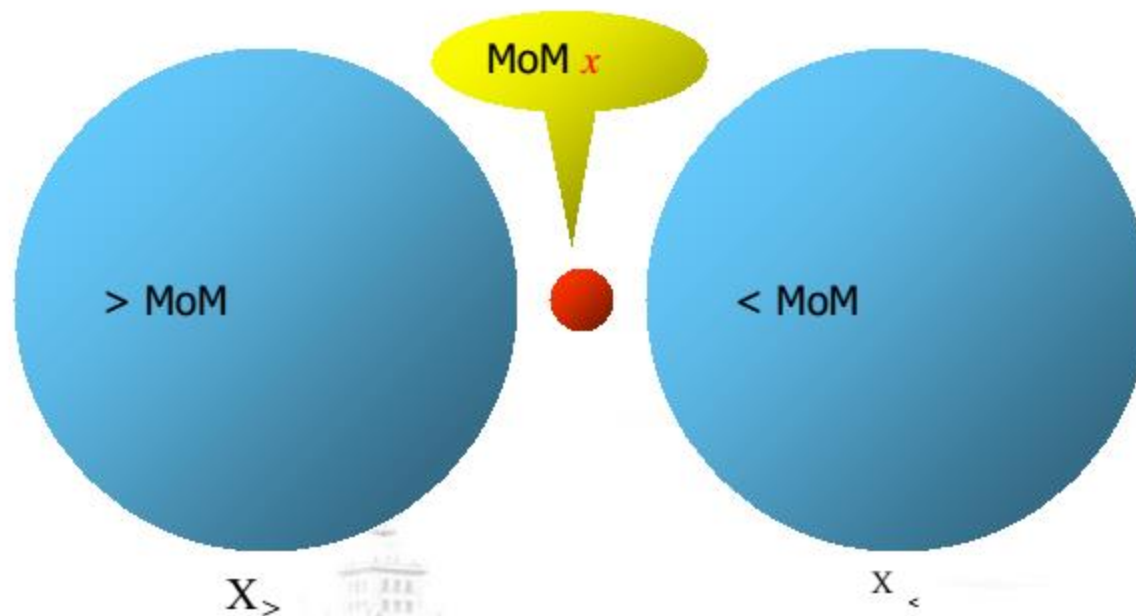
第三步：递归调用算法求得这些中位数的中位数(MoM)



时间复杂性: $T(\lfloor n/5 \rfloor)$



第四步:用 MoM 完成划分



时间复杂性 $O(n)$





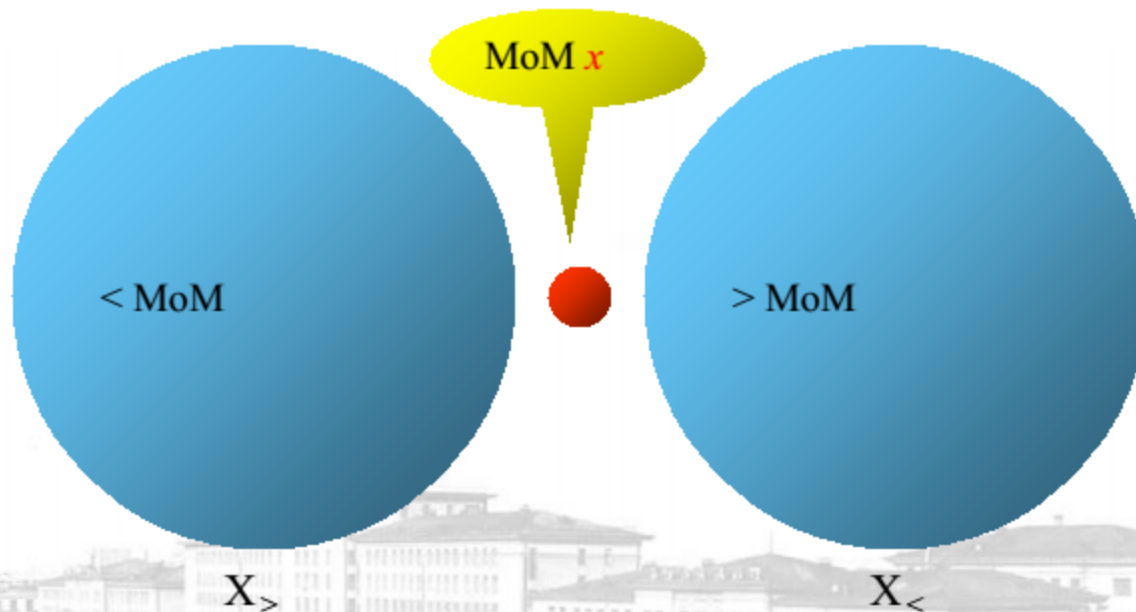
第五步:递归

设 x 是中位数的中位数(MoM),划分完成后其下标为 k

如果 $i=k$,则返回 x

如果 $i < k$,则在第一个部分递归选取第 i -大的数

如果 $i > k$,则在第三个部分递归选取第 $(i-k)$ -大的数





算法Select(A, i)

Input: 数组 $A[1:n]$, $1 \leq i \leq n$

Output: $A[1:n]$ 中的第 i -大的数

1. for $j \leftarrow 1$ to $n/5$ ← 第一步
2. InsertSort($A[(j-1)*5+1 : (j-1)*5+5]$); } 第二步
3. swap($A[j]$, $A[(j-1)*5+3]$);
4. $x \leftarrow \text{Select}(A[1: n/5], n/10)$; ← 第三步
5. $k \leftarrow \text{partition}(A[1:n], x)$; ← 第四步
6. if $k=i$ then return x ;
7. else if $k>i$ then retrun Select($A[1:k-1], i$); } 第五步
8. else retrun Select($A[k+1:n], i-k$);





算法分析

算法 $\text{Select}(A, i)$

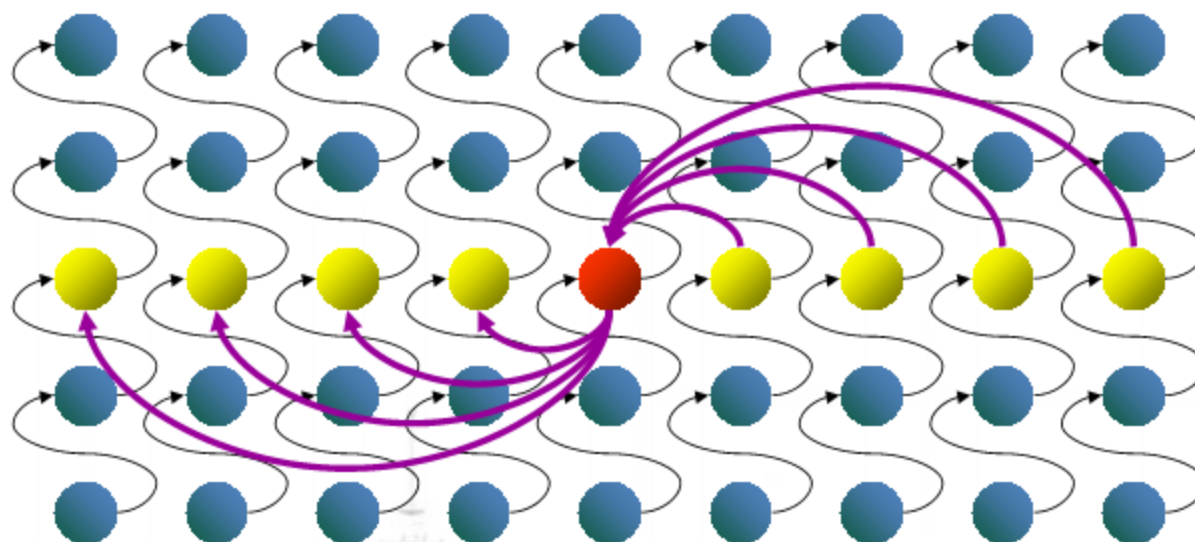
Input: 数组 $A[1:n]$, $1 \leq i \leq n$

Output: $A[1:n]$ 中的第 i -大的数

1. for $j \leftarrow 1$ to $n/5$
 2. $\text{InsertSort}(A[(j-1)*5+1 : (j-1)*5+5]);$
 3. $\text{swap}(A[j], A[(j-1)*5+3]);$
 4. $x \leftarrow \text{Select}(A[1: n/5], n/10);$
 5. $k \leftarrow \text{partition}(A[1:n], x);$
 6. if $k=i$ then return x ;
 7. else if $k>i$ then retrun $\text{Select}(A[1:k-1], i);$
 8. else retrun $\text{Select}(A[k+1:n], i-k);$
- Annotations:
- Lines 2 and 3 are grouped by a brace and labeled $O(n)$.
 - Line 4 is labeled $T(\lfloor n/5 \rfloor)$.
 - Line 5 is labeled $O(n)$.
 - Lines 6, 7, and 8 are grouped by a brace and labeled ???.

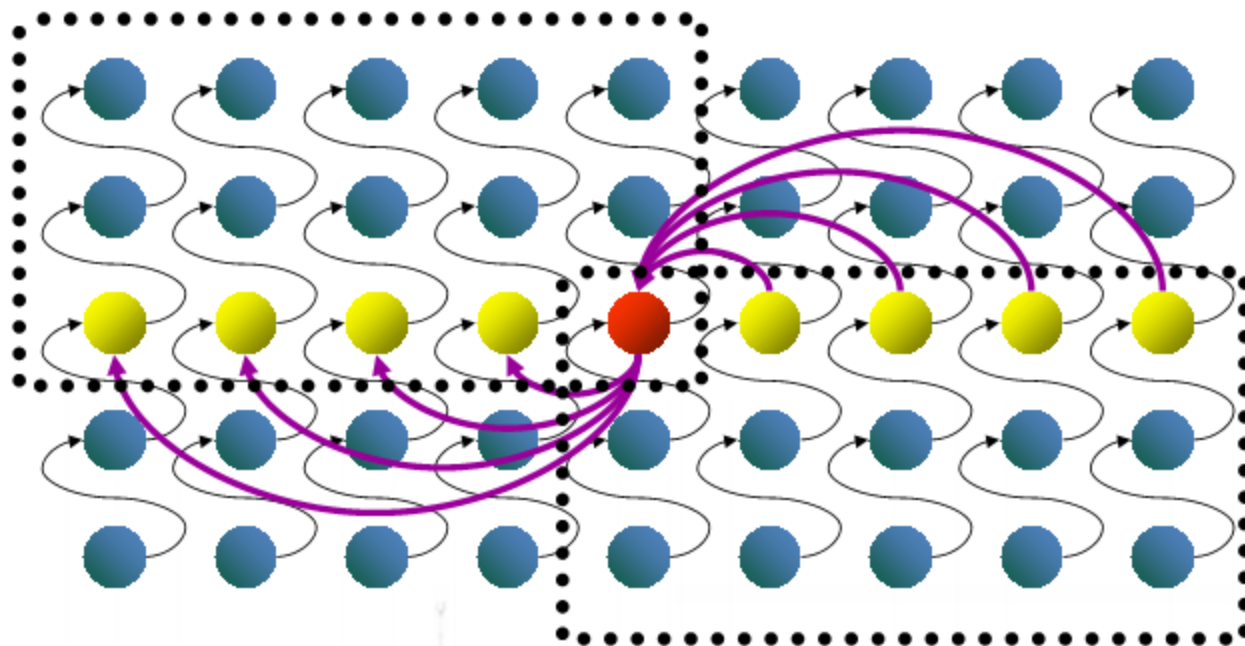


观察第五步的处理过程





第五步至少删除了 $\lfloor 3n/10 \rfloor$ 个数



$$n - \lfloor 3n/10 \rfloor \leq 7n/10 + 6$$

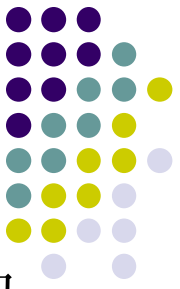
如果时间复杂度是输入规模的递增函数
则第五步的时间开销不超过 $T(7n/10 + 6)$



$$T(n) \leq \begin{cases} \Theta(1) & \text{if } n \leq C \\ T\lfloor n/5 \rfloor + T(7n/10 + 6) + \Theta(n) & \text{if } n > C \end{cases}$$

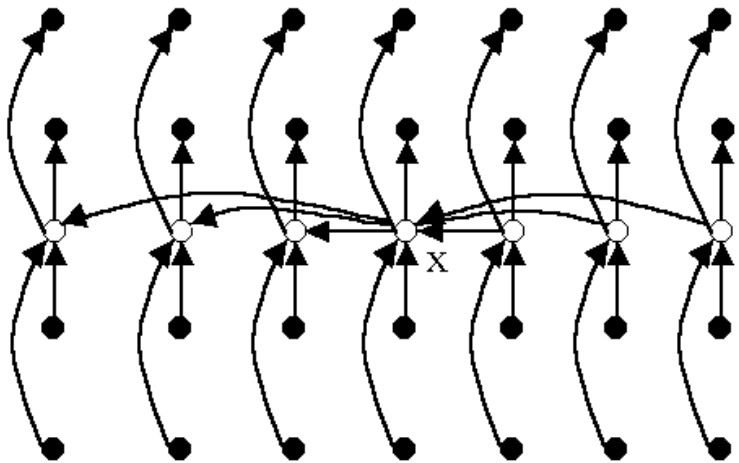
$$T(n) = O(n)$$



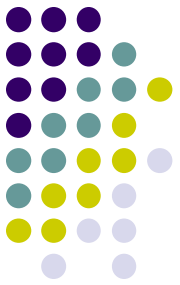


线性时间选择

- 将 n 个输入元素划分成 $\lceil n/5 \rceil$ 个组，每组5个元素，只可能有一个组不是5个元素。用任意一种排序算法，将每组中的元素排好序，并取出每组的中位数，共 $\lceil n/5 \rceil$ 个。
- 递归调用**select**来找出这 $\lceil n/5 \rceil$ 个元素的中位数。如果 $\lceil n/5 \rceil$ 是偶数，就找它的2个中位数中较大的一个。以这个元素作为划分基准。



设所有元素互不相同。在这种情况下，找出的基准 x 至少比 $3(n-5)/10$ 个元素大，因为在每一组中有2个元素小于本组的中位数，而 $n/5$ 个中位数中又有 $(n-5)/10$ 个小于基准 x 。同理，基准 x 也至少比 $3(n-5)/10$ 个元素小。而当 $n \geq 75$ 时， $3(n-5)/10 \geq n/4$ 所以按此基准划分所得的2个子数组的长度都至少缩短 $1/4$ 。



```
Type Select(Type a[], int p, int r, int k)
```

```
{
```

```
    if (r-p<5) {
```

```
        用某个简单排序算法对数组 a[p:r] 排序
```

```
        ret
```

```
    };
```

```
    for (
```

```
        将
```

```
        与
```

复杂度分析

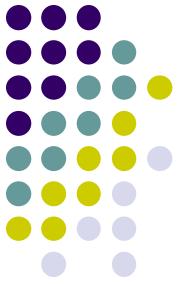
$$T(n) \leq \begin{cases} C_1 & n < 75 \\ C_2 n + T(n/5) + T(3n/4) & n \geq 75 \end{cases}$$

$$T(n) = O(n)$$

```
//找中位数的中位数， r-p-4即上面所说的n-5
```

上述算法将每一组的大小定为5，并选取75作为是否作递归调用的分界点。这2点保证了 $T(n)$ 的递归式中2个自变量之和 $n/5 + 3n/4 = 19n/20 = \epsilon n$ ， $0 < \epsilon < 1$ 。这是使 $T(n) = O(n)$ 的关键之处。当然，除了5和75之外，还有其他选择。

```
}
```



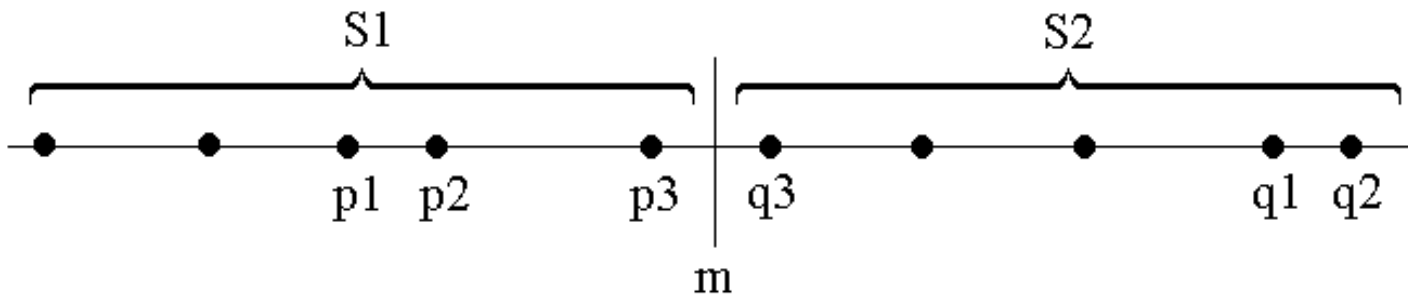
最接近点对问题



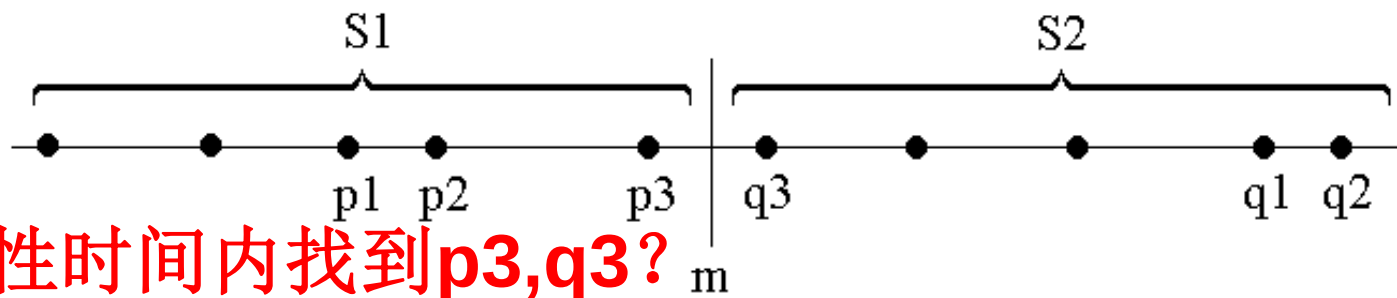
最接近点对问题

◆为了使问题易于理解和分析，先来考虑一维的情形。此时， S 中的 n 个点退化为 x 轴上的 n 个实数 $x_1, x_2, \dots, x_n$ 。最接近点对即为这 n 个实数中相差最小的2个实数。

- 假设我们用 x 轴上某个点 m 将 S 划分为2个子集 S_1 和 S_2 ，基于**平衡子问题**的思想，用 S 中各点坐标的中位数来作分割点。
- 递归地在 S_1 和 S_2 上找出其最接近点对 $\{p_1, p_2\}$ 和 $\{q_1, q_2\}$ ，并设 $d = \min\{|p_1 - p_2|, |q_1 - q_2|\}$ ， S 中的最接近点对或者是 $\{p_1, p_2\}$ ，或者是 $\{q_1, q_2\}$ ，或者是某个 $\{p_3, q_3\}$ ，其中 $p_3 \in S_1$ 且 $q_3 \in S_2$ 。
- 能否在线性时间内找到 p_3, q_3 ？



最接近点对问题



能否在线性时间内找到 $p3, q3$?

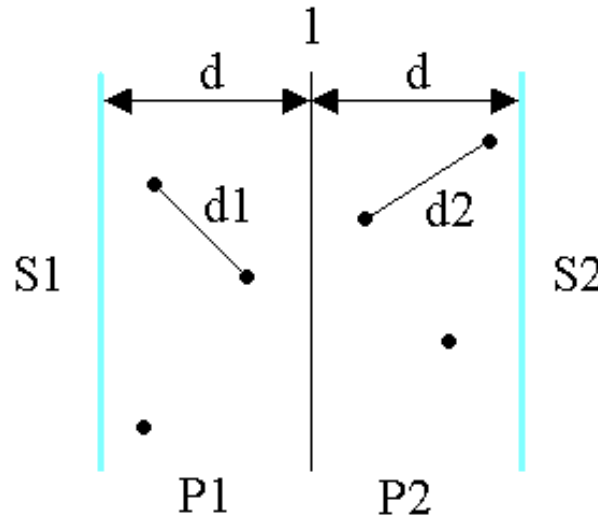
- ◆如果 S 的最接近点对是 $\{p3, q3\}$, 即 $|p3 - q3| < d$, 则 $p3$ 和 $q3$ 两者与 m 的距离不超过 d , 即 $p3 \in (m-d, m]$, $q3 \in (m, m+d]$ 。
- ◆由于在 $S1$ 中, 每个长度为 d 的半闭区间至多包含一个点 (否则必有两点距离小于 d), 并且 m 是 $S1$ 和 $S2$ 的分割点, 因此 $(m-d, m]$ 中至多包含 S 中的一个点。由图可以看出, 如果 $(m-d, m]$ 中有 S 中的点, 则此点就是 $S1$ 中最大点。
- ◆因此, 我们用线性时间就能找到区间 $(m-d, m]$ 和 $(m, m+d]$ 中所有点, 即 $p3$ 和 $q3$ 。从而我们用线性时间就可以将 $S1$ 的解和 $S2$ 的解合并成为 S 的解。



最接近点对问题

◆ 下面来考虑二维的情形。

- 选取一垂直线 $l: x=m$ 来作为分割直线。其中 m 为 S 中各点 x 坐标的中位数。由此将 S 分割为 S_1 和 S_2 。
- 递归地在 S_1 和 S_2 上找出其最小距离 d_1 和 d_2 ，并设 $d = \min\{d_1, d_2\}$ ， S 中的最接近点对或者是 d ，或者是某个 $\{p, q\}$ ，其中 $p \in P_1$ 且 $q \in P_2$ 。
- 能否在线性时间内找到 p, q ？

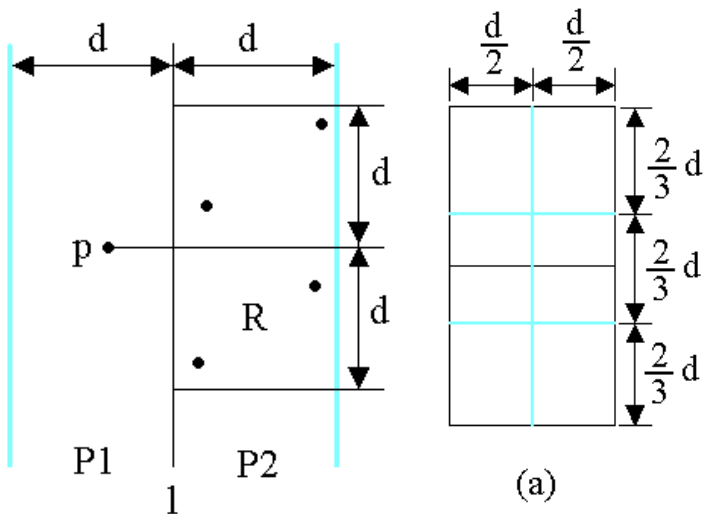




最接近点对问题

能否在线性时间内找到 p_3, q_3 ?

- 考虑 P_1 中任意一点 p ，它若与 P_2 中的点 q 构成最接近点对的候选者，则必有 $\text{distance}(p, q) < d$ 。满足这个条件的 P_2 中的点一定落在一个 $d \times 2d$ 的矩形 R 中
- 由 d 的意义可知， P_2 中任何2个 S 中的点的距离都不小于 d 。由此可以推出矩形 R 中最多只有6个 S 中的点。
- 因此，在分治法的合并步骤中最多只需要检查 $6 \times n/2 = 3n$ 个候选者



证明:将矩形 R 的长为 $2d$ 的边3等分，将它的长为 d 的边2等分，由此导出6个 $(d/2) \times (2d/3)$ 的矩形。若矩形 R 中有多于6个 S 中的点，则由鸽舍原理易知至少有一个 $(d/2) \times (2d/3)$ 的小矩形中有2个以上 S 中的点。设 u, v 是位于同一小矩形中的2个点，则

$$(x(u) - x(v))^2 + (y(u) - y(v))^2 \leq (d/2)^2 + (2d/3)^2 = \frac{25}{36}d^2$$

$\text{distance}(u, v) < d$ 。这与 d 的意义相矛盾。



最接近点对问题

- 为了确切地知道要检查哪6个点，可以将 p 和 $P2$ 中所有 $S2$ 的点投影到垂直线 l 上。由于能与 p 点一起构成最接近点对候选者的 $S2$ 中点一定在矩形 R 中，所以它们在直线 l 上的投影点距 p 在 l 上投影点的距离小于 d 。由上面的分析可知，这种投影点最多只有6个。
- 因此，若将 $P1$ 和 $P2$ 中所有 S 中点按其 y 坐标排好序，则对 $P1$ 中所有点，对排好序的点列作一次扫描，就可以找出所有最接近点对的候选者。对 $P1$ 中每一点最多只要检查 $P2$ 中排好序的相继6个点。



最接近点对问题

```
double cpair2(S)
{
    n=|S|;
    if (n < 4)
        return cpair1(S);
    1、 m=S中点的x坐标的中位数;
    构造S1={p∈S|x(p)≤m}, S2={p∈S|x(p)>m}
    //S1={p∈S|x(p)≤m},
    S2={p∈S|x(p)>m}
    2、 d1=cpair2(S1);
    d2=cpair2(S2);
    3、 dm=min(d1,d2);
    4、 设P1是S1中距垂直分割线l的距离在dm之内的所有点组成的集合;
    P2是S2中距分割线l的距离在dm之内所有点组成的集合;
    5、 在P1与P2中找出距离最小的点对d1;
    6、 d=min(dm,d1);
    return d;
}
```

复杂度分析

$$T(n) = \begin{cases} O(1) & n < 4 \\ 2T(n/2) + O(n) & n \geq 4 \end{cases}$$

$$T(n)=O(n\log n)$$

中与
完成

当X中的扫描指针逐次向上移动时，Y中的扫描指针可在宽为 $2dm$ 的区间内移动；

设 $d1$ 是按这种扫描方式找到的点对间的最小距离；

6、 $d=\min(dm,d1)$;
return d;

}



循环赛日程表



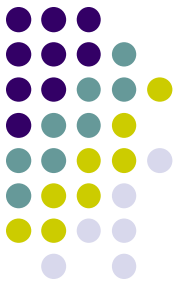
循环赛日程表

设计一个满足以下要求的比赛日程表：

- (1) 每个选手必须与其他 $n-1$ 个选手各赛一次；
- (2) 每个选手一天只能赛一次；
- (3) 循环赛一共进行 $n-1$ 天。

按分治策略，将所有的选手分为两半， n 个选手的比赛日程表就可以通过为 $n/2$ 个选手设计的比赛日程表来决定。递归地用对选手进行分割，直到只剩下2个选手时，比赛日程表的制定就变得很简单。这时只要让这2个选手进行比赛就可以了。

1	2	3	4	5	6	7	8
2	1	4	3	6	5	8	7
3	4	1	2	7	8	5	6
4	3	2	1	8	7	6	5
5	6	7	8	1	2	3	4
6	5	8	7	2	1	4	3
7	8	5	6	3	4	1	2
8	7	6	5	4	3	2	1



课堂任务

- 算法实现题1:
 - 在向量 $V[\text{first}, \text{last}]$ 中查找出第 K 大元素的值
- 算法实现题2:
 - 2-33, 2-34